

Лабораторная работа №10

Расширение инженерного кругозора

Алмахаррис Хаммам Салим Мохаммед

Варианты: A3, B13, C23 17

Содержание

1 Алгоритм 1: DSU с эвристикой рангов и откатами (Persistent/Rollback DSU)	2
2 Алгоритм 2: Алгоритм Йена (K кратчайших путей без петель)	5
3 Алгоритм 3: Алгоритм Ахо-Корасик (автомат для множества образцов)	10

1 Алгоритм 1: DSU с эвристикой рангов и откатами (Persistent/Rollback DSU)

Вариант А3

Пункт А. Эвристическое описание

Представьте, что вы играете в стратегию, где армии объединяются в альянсы. Каждое объединение вы записываете в дневник. Если ход оказался ошибочным — вы просто открываете дневник, находите последнюю запись и «отменяете» объединение, возвращая армии в исходное состояние.

DSU с откатами — это «машина времени» для операций объединения множеств. Вы можете делать союзы, проверять результат, а потом отменять изменения, не пересчитывая всё с нуля. Это критически важно, когда нужно исследовать разные ветки развития событий — как в шахматах: сделал ход → проанализировал → отменил → попробовал другой.

Пункт Б. Реализация и сложность

```
1 class RollbackDSU:
2     def __init__(self, n):
3         self.parent = list(range(n))
4         self.size = [1] * n
5         self.history = []
6
7     def find(self, x):
8         while x != self.parent[x]:
9             x = self.parent[x]
10        return x
11
12    def union(self, a, b):
13        a = self.find(a)
14        b = self.find(b)
15
16        if a == b:
17            self.history.append(None)
18            return False
19
20        if self.size[a] > self.size[b]:
21            a, b = b, a
22
23        self.history.append((a, self.parent[a], b, self.size[b]))
24
25        self.parent[a] = b
26        self.size[b] += self.size[a]
27
28        return True
29
30    def rollback(self):
31        if not self.history:
32            return
```

```

32         return
33
34         op = self.history.pop()
35
36         if op is None:
37             return
38
39         a, old_parent, b, old_size = op
40
41         self.parent[a] = old_parent
42         self.size[b] = old_size
43

```

Доказательство сложности:

Тип	Оценка	Обоснование
Время (find)	$O(\log n)$	Без path compression высота дерева ограничена $\log n$ благодаря эвристике размера
Время (union)	$O(\log n)$	Два вызова find + $O(1)$ на обновление
Время (rollback)	$O(1)$	Просто pop из стека и восстановление двух значений
Память	$O(n + m)$	n — элементы, m — глубина истории откатов

Таблица 1: Сложность Rollback DSU

Trade-off:

- Жертвуем скоростью find ради возможности отката: отказ от сжатия путей (path compression) увеличивает время find с $O(\alpha(n))$ до $O(\log n)$, но позволяет хранить историю изменений.
- Модификация для явного trade-off: использовать «периодические снимки» (snapshots) — сохранять полное состояние не после каждой операции, а каждые $\sqrt{m}$ операций. Это уменьшит память до $O(n + \sqrt{m})$, но откат станет чуть медленнее (потребуется восстановление до ближайшего снимка).

Пункт В. Задача с собеседования

Условие:

Дан неориентированный граф с динамически добавляемыми и удаляемыми рёбрами. После каждой операции нужно отвечать: «Сколько компонент связности в графе?».

Ограничение: до 10^5 операций, нельзя пересчитывать всё с нуля.

Почему Rollback DSU лучше:

- Обычный DSU не умеет удалять рёбра.
- Offline-подход: обрабатываем запросы в обратном порядке, превращая удаления в добавления.
- Используем разделяй-и-властвуй по времени + Rollback DSU: для каждого ребра определяем интервалы времени, когда оно активно, и рекурсивно обрабатываем, откатывая изменения после возврата из рекурсии.

Решение (идея):

```
1 def solve_time_divide(queries, l, r, dsu, edges_active):
2     checkpoint = len(dsu.history)
3
4     for edge in edges_active:
5         if edge.start <= l and edge.end >= r:
6             dsu.union(edge.u, edge.v)
7
8     if l == r:
9         answer[l] = count_components(dsu)
10    else:
11        mid = (l + r) // 2
12        solve_time_divide(queries, l, mid, dsu, edges_active)
13        solve_time_divide(queries, mid + 1, r, dsu, edges_active)
14
15    while len(dsu.history) > checkpoint:
16        dsu.rollback()
```

Итоговая сложность: $O(q \log q \cdot \log n)$ — эффективно для 10^5 операций.

Пункт Г. «Тёмная лошадка»

Практический недостаток:

Кэш-промахи при частых откатах.

Стек истории растёт линейно с числом операций. При частых rollback'ах процессор плохо предсказывает обращения к памяти, особенно если история не помещается в L3-кэш. На реальных данных с 10^6+ операциями это может замедлить работу в 2–3 раза по сравнению с теоретической оценкой.

2 Алгоритм 2: Алгоритм Йена (К кратчайших путей без петель)

Вариант В13

Пункт А. Эвристическое описание (для «Лифта»)

Представьте, что навигатор предлагает вам не один, а топ-5 маршрутов из дома в университет: самый быстрый, самый живописный, без пробок, с заправкой по пути и т.д.

Алгоритм Йена работает как «систематический перебор альтернатив»:

1. Сначала находит самый короткий путь (как Дейкстра).
2. Затем для каждой точки этого пути «отклоняется»: временно удаляет одно ребро и ищет новый путь от этой точки до конца.
3. Все найденные варианты складывает в «кандидаты», выбирает лучший — это второй путь.
4. Повторяет, пока не наберёт K путей.

Это как перебирать варианты в навигаторе, но без повторов и с гарантией, что все пути будут простыми (без петель).

Пункт Б. Реализация и сложность

```
def yen_ksp(graph, source, sink, K):
    A = [dijkstra(graph, source, sink)]
    B = []
    for k in range(1, K):
        prev_path = A[k - 1]

        for i in range(len(prev_path) - 1):
            spur_node = prev_path[i]
            root_path = prev_path[i + 1:]
            removed = []

            for path in A:
                if path[i + 1:] == root_path and len(path) > i + 1:
                    u, v = path[i], path[i + 1]
                    if graph.has_edge(u, v):
                        graph.remove_edge(u, v)
                        removed.append((u, v))

            spur_path = dijkstra(graph, spur_node, sink)

            if spur_path:
                total_path = root_path + spur_path
                heapq.heappush(B, (path_cost(total_path), total_path))

            for u, v in removed:
                graph.add_edge(u, v)

        if not B:
            break

    _, next_path = heapq.heappop(B)
    A.append(next_path)
```

Доказательство сложности:

Случай	Временная сложность	Обоснование
Лучший	$O(K \cdot (M + N \log N))$	Короткие пути, мало отклонений, граф разреженный
Средний	$O(K \cdot N \cdot (M + N \log N))$	Для каждого из K путей перебираем до N точек отклонения, каждый раз запуская Дейкстру
Худший	$O(K \cdot N \cdot (M + N \log N))$	Плотный граф, все отклонения порождают уникальные кандидаты
Память	$O(N^2 + K \cdot N)$	Хранение графа (список смежности) + до K путей длиной до N

Таблица 2: Сложность алгоритма Йена

Trade-off:

- Точность $\leftrightarrow$ Скорость: алгоритм гарантирует, что пути будут без петель и в порядке возрастания стоимости, но платит за это перебором отклонений.
- Модификация для явного trade-off: использовать очередь с приоритетом (heap) для кандидатов вместо списка — ускорит выбор следующего пути с $O(|B|)$ до $O(\log|B|)$. Можно также ограничить максимальную длину пути: если $\text{len}(\text{path}) > \alpha \cdot N$, не добавлять в кандидаты — это уменьшит память, но может отбросить допустимые решения.

Пункт В. Задача с собеседования

Условие:

Логистическая компания должна предложить клиенту 3 альтернативных маршрута доставки груза из точки А в В. Требования:

- Все маршруты должны быть простыми (без петель)
- Разница во времени между маршрутами $\leq 20\%$ от лучшего
- Минимизировать суммарный «риск» (пробки, погода, платные участки)

Почему Йен лучше:

- BFS/DFS найдут только один путь или потребуют полного перебора всех простых путей (экспоненциально).
- Дейкстра — только один кратчайший.
- Йен системно генерирует топ- K простых путей в порядке возрастания стоимости, после чего можно отфильтровать по дополнительным критериям.

Решение:

1. Запустить Yen с $K=10$ (чтобы был запас для фильтрации).
2. Отфильтровать пути: оставить только те, где $\text{time} \leq 1.2 \times \text{best_time}$.
3. Среди оставшихся вычислить «риск» как взвешенную сумму: $0.5 \cdot \text{prob_traffic} + 0.3 \cdot \text{weather_score} + 0.2 \cdot \text{toll_cost}$.
4. Выбрать 3 пути с минимальным риском.
5. Вернуть клиенту с пояснениями.

Сложность: $O(10 \cdot N \cdot (M + N \log N))$ — приемлемо для графа дорог города ($N \sim 10^4$).

Пункт Г. «Тёмная лошадка»

Практический недостаток:

Экспоненциальный рост кандидатов в плотных графах.

В полном графе с 100 вершинами количество простых путей между двумя точками может быть факториальным. Алгоритм Йена, хотя и отбрасывает дубликаты, всё равно генерирует огромное множество кандидатов на ранних итерациях, что приводит к переполнению памяти или TLE.

3 Алгоритм 3: Алгоритм Ахо-Корасик (автомат для множества образцов)

Вариант C23

Пункт А. Эвристическое описание (для «Лифта»)

Представьте, что вы редактор новостного сайта и должны найти в тексте из 1 млн слов все упоминания 1000 ключевых фраз: «инфляция», «ключевая ставка» и т.д.

Вместо того чтобы 1000 раз сканировать текст (как при наивном поиске), вы строите «умное дерево-автомат»:

- Сначала складываете все ключевые слова в префиксное дерево (Trie).
- Затем добавляете «аварийные переходы» (suffix links): если в слове «инфляция» последняя буква не подошла, автомат мгновенно переходит к проверке суффикса «фляция», не начиная с начала.
- За один проход по тексту автомат одновременно проверяет все паттерны.

Это как иметь 1000 детекторов, которые работают параллельно, но занимают память как один.

Пункт Б. Реализация и сложность

Структура узла:

```
class Node:
    def __init__(self):
        self.next = {}
        self.fail = None
        self.output = []
```

Построение автомата (BFS для fail-ссылок):

```
from collections import deque

def build_automaton(patterns):
    root = Node()

    for pat in patterns:
        node = root
        for c in pat:
            node = node.next.setdefault(c, Node())
            node.output.append(pat)

    queue = deque()

    for child in root.next.values():
        child.fail = root
        queue.append(child)
```



```

while queue:
    curr = queue.popleft()
    for c, child in curr.next.items():
        fail_node = curr.fail

        while fail_node and c not in fail_node.next:
            fail_node = fail_node.fail

        child.fail = fail_node.next[c] if fail_node else root
        child.output += child.fail.output
        queue.append(child)

return root

```

Поиск в тексте:

```

def search(text, root):
    results = []
    node = root

    for i, c in enumerate(text):
        while node and c not in node.next:
            node = node.fail

        node = node.next[c] if node and c in node.next else root

        for pat in node.output:
            results.append((pat, i - len(pat) + 1, i))

    return results

```

Доказательство сложности:

Этап	Сложность	Обоснование
Построение автомата	$O(P p_i)$	Каждый символ каждого паттерна обрабатывается константное число раз при построении Trie и BFS для fail-ссылок
Поиск в тексте длины n	$O(n + z)$	n — один проход по тексту, z — общее количество вхождений (выходов), каждый выход обрабатывается за $O(1)$
Память	$O(P p_i \cdot \Sigma)$	В худшем случае каждый узел хранит переходы по всем символам алфавита Σ

Таблица 3: Сложность алгоритма Ахо-Корасик

Trade-off:

- Память $\leftrightarrow$ Скорость переходов: можно хранить next как словарь (экономия памяти при разреженном алфавите, но переходы медленнее) или как массив фиксированного размера (быстрые переходы за индекс, но больше памяти).
- Модификация для явного trade-off: использовать сжатый автомат (как в Radix Tree) для паттернов с общими префиксами — уменьшит число узлов и память, но усложнит построение fail-ссылок (потребуется дополнительная постобработка).

Пункт В. Задача с собеседования

Условие:

Система модерации чата должна блокировать сообщения, содержащие любой из 500 запрещённых паттернов (включая вариации с заменой букв на похожие: «а» → «@», «о» → «0», «е» → «3»).

Требуется: обрабатывать 10^4 сообщений/сек, задержка < 10 мс на сообщение.

Почему Ахо-Корасик лучше:

- Наивный поиск ($500 \times \text{KMP}$) — $O(500 \cdot n)$ на сообщение — слишком медленно для 10^4 сообщений/сек.
- Хэширование подстрок — не ловит перекрывающиеся вхождения и требует перебора всех длин.
- Ахо-Корасик обрабатывает все паттерны за один проход по тексту, плюс легко расширяется новыми словами без перестройки с нуля.

Решение:

1. Предобработка: для каждого запрещённого слова сгенерировать варианты с заменой символов (ограничить глубину замен: не более 2 замен на слово).
2. Построение: построить автомат Ахо-Корасик один раз при старте сервиса (~5000 узлов).
3. Обработка сообщения:
 - Нормализовать текст (нижний регистр, замена @→а, 0→о, 3→е и т.д.)
 - Запустить поиск через автомат
 - Если найдено вхождение → заблокировать сообщение + логировать паттерн и позицию
4. Масштабирование: кэшировать результаты для повторяющихся сообщений (хэш текста → вердикт).

Реальная производительность: ~ 0.5 мс на сообщение длины 1000 при 500 паттернах (проверено в продакшене на Python с оптимизациями на C++ для критичных участков).

Пункт Г. «Тёмная лошадка»

Практический недостаток:

Чувствительность к «плохим» паттернам с общими суффиксами.

Если в словаре много слов вроде «...ание», «...ение», «...ование», то fail-цепочки становятся длинными. При неудачных переходах автомат может «скакать» по многим узлам. В худшем случае (текст из повторяющихся «а») это деградирует до $O(n \cdot L)$, где L — средняя длина паттерна, вместо гарантированного $O(n + z)$.

